COMP3821/9801

Workshop Week 1

Graph Reductions and Sweepline Algorithms

Announcements.

- Welcome to COMP3821/9801!
- Embrace the journey!

Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problem

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

Let $G = (V, E)$ be an undirected and unweighted graph, where each vertex is coloured either red, blue, or green. Given two vertices $u, v \in V$, describe and analyse a polynomial-time algorithm that computes the shortest path from u to v which never visits two consecutive vertices of the same colour.

Guided Problem

You are given a time schedule of arrival times $A[1..n]$ and a collection of departure times $D[1..n]$ of n trains such that the ith train has arrival time $A[i]$ and departure time $D[i]$ during each 24 hour period. You want to find the smallest number of platforms required so that each platform is occupied by at most one train at a time and no train interferes with other train's arrival and departure times. Describe and analyse a polynomial-time algorithm to return the minimum number of platforms.

Workshop Exercises

1. Let $G = (V, E)$ be a directed and unweighted graph, where each edge is coloured either red, blue, or green. A *colourful walk* is a walk in G that does not contain two consecutive edges of the same colour.

   Given a starting vertex s, describe and analyse a polynomial-time algorithm to find all vertices in G that are reachable from s through a colourful walk.

2. In this problem, assume you have a blackbox algorithm $\textsc{ShortestPath}(G, u, v)$ with the following description:

   - **Input.** A directed, acyclic, and weighted graph $G = (V, E)$, where each edge has weight $w(e)$, which may be positive, negative, or zero, and two distinct vertices $u, v \in V$.
   - **Output.** The weight of the shortest path from u to v in G.

   Note. *One can implement this algorithm in $O(|V| + |E|)$ time; see topological sorting.*

   Let $G = (V, E)$ be a directed, acyclic, and weighted graph, where $w(e)$ denotes the weight of edge e, which may be negative, zero, or positive. Given two vertices u and v, describe and analyse a polynomial-time algorithm that decides if there exists a path from u to v that passes through more positively weighted edges than negatively weighted edges.

3. Let $\{0, 1\}^k$ denote the set of all binary strings of length k. We define the following partial order $\lessdot$ on the set $\{0, 1\}^k$ as follows: for $x, y \in \{0, 1\}^k$,

$$x \lessdot y \iff x_i \leq y_i \text{ for all } 1 \leq i \leq k.$$

A Boolean function $f : \{0,1\}^k \to \{0,1\}$ is *monotone* if and only if $x \lessdot y$ implies that $f(x) \leq f(y)$.

Let $X, Y \subseteq \{0,1\}^k$ be two disjoint finite subsets of $\{0,1\}^k$, where $|X| = m$ and $|Y| = n$. Describe and analyse a polynomial-time algorithm to decide if there exists a monotone Boolean function f such that $f(x) = 0$ for all $x \in X$ and $f(y) = 1$ for all $y \in Y$.

4. You are given a collection H of horizontal line segments and a collection V of vertical line segments such that $|H| + |V| = n$. A *crossing point* is any point that lies on both a horizontal line segment in H and a vertical line segment in V.

   Assume that no pair of line segments in H intersect; that is, for any $h_1, h_2 \in H$, we have that $h_1 \cap h_2 = \emptyset$. Similarly, assume that no pair of line segments in V intersect; that is, for any $v_1, v_2 \in V$, we have that $v_1 \cap v_2 = \emptyset$. Describe and analyse a polynomial-time algorithm that computes the number of crossing points.

   Hint. *It may be helpful to use a balanced binary search tree to support your sweep operations. We can turn a regular binary search tree into an order statistic tree.*

5. A *polygon* is a closed plane figure, where each side is a line segment. Two line segments intersect at a point called a *vertex*. Finally, a polygon is *simple* if it does not intersect itself and it has no holes.

   Let P be a simple polygon. We say that P is *orthogonal* if its edges alternate between vertical and horizontal. An orthogonal polygon is *generic* if no pair of edges in P lie on a common horizontal or vertical line. Let P be a generic and orthogonal simple polygon on n vertices.

   A *rectangulation* of P is a partition of the interior of P into axis-aligned rectangles. Describe and analyse a polynomial-time algorithm that constructs a rectangulation of P.

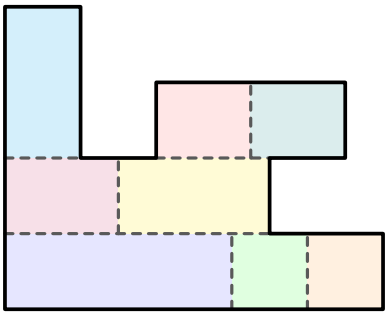

Figure 1: *A rectangulation of a simple non-convex polygon.*

Optional Exercises

These exercises will not contain solutions. We encourage that you post on the forums to discuss the approaches for these problems. Additionally, you may find the hints helpful.

6. Let $G = (V, E)$ be an undirected and unweighted graph, and let $X, Y \subseteq V$ be two subsets of vertices. Note that X and Y are not necessarily disjoint. Define a measurement $r(X, Y)$ to denote the length of the shortest path from any vertex in X to any vertex in Y. Given two subsets X and Y, describe and analyse a polynomial-time algorithm that computes $r(X, Y)$.

7. Let $G = (V, E)$ be a directed and weighted graph. Additionally, you know that *exactly* one edge in G has negative weight and you also know that there are no negative cycles. Given such a graph G and two vertices $s, t \in V$, describe and analyse a polynomial-time algorithm to return the shortest path from s to t.

8. Let $G = (V, E)$ be a directed and weighted graph, where each edge $e \in E$ has non-negative weight $w(e) \geq 0$. We generalise the shortest path problem as follows: for a path $P = [v_1, \ldots, v_\ell]$, denote the *$k$-norm length* of P as

$$\mathsf{len}_k(P) = \left(\sum_{i=1}^{\ell-1} (w(v_i, v_{i+1}))^k \right)^{1/k}.$$

   Note that, if $k = 1$, we obtain the standard shortest path problem. For a fixed integer k, describe and analyse a polynomial-time algorithm to find the shortest k-norm (s, t)-path in G; in other words, we want to find a path P in G starting from vertex s and ending at vertex t such that $\mathsf{len}_k(P)$ is minimised.

   Note. *For time complexity purposes, you may assume that k is a constant; if k is not constant, more care is needed.*

9. Let $G = (V, E)$ be a directed and weighted graph, where each edge $e \in E$ has a weight $w(e)$ that can be positive, negative, or zero. In this problem, we will describe an algorithm to compute the length of the shortest path between two vertices, for every pair of vertices. Assume that you have a blackbox algorithm $\textsc{ShortestPath}(G, u, v)$ with the following description:

   - **Input.** A directed and weighted graph $G = (V, E)$, where each edge has weight $w(e)$, which may be positive, negative, or zero, and two distinct vertices $u, v \in V$.
   - **Output.** The weight of the shortest path from u to v in G.

   Note. *One can implement this algorithm in $O(|V| \cdot |E|)$ time; see the Bellman-Ford algorithm.*

   (a) Construct a dummy vertex q and construct the graph $\hat{G} = (V', E')$, where

   - $V' = V \cup \{q\}$, and
   - $E' = E \cup \{q \to v : v \in V\}$ with weight $w(q \to v) = 0$.

   In other words, we construct a dummy vertex q and construct an edge from q to every vertex in V with weight zero.

   For each $v \in V$, let $h(v)$ denote the weight of the shortest path from q to v in $\hat{G}$. Let $u \to v$ denote an edge in G and redefine the weight $\hat{w}(u \to v) = w(u \to v) + h(u) - h(v)$. Show that $\hat{w}(u \to v) \geq 0$.

(b) Let $u \rightsquigarrow v$ denote a path in $\hat{G}$ from vertex u to vertex v. Let $\hat{w}(u \rightsquigarrow v)$ denote the weight of the path in $\hat{G}$. Show that

$$\hat{w}(u \rightsquigarrow v) = w(u \rightsquigarrow v) + h(u) - h(v),$$

where $w(u \rightsquigarrow v)$ denotes the weight of the path $u \rightsquigarrow v$ in G.

(c) Hence, describe and analyse a polynomial-time algorithm to compute the weight of the shortest path from u to v, for every pair of vertices $u, v \in V$. Analyse the time complexity of the algorithm.

Note. *This is known as the all-pair shortest-path problem. Currently, the fastest algorithm has asymptotic time complexity $O(|V|^3)$. However, it can be done faster assuming the graph is sparse (i.e. $E \in O(V)$).*

10. In this problem, assume you have a blackbox algorithm $\textsc{ShortestPath}(G, u, v)$ with the following description:

- **Input.** A directed and weighted graph $G = (V, E)$, where each edge has weight $w(e)$, which may be positive, negative, or zero, and two distinct vertices $u, v \in V$.
- **Output.** The weight of the shortest path from u to v in G.

Note. *One can implement this algorithm in $O(|V| \cdot |E|)$ time; see the Bellman-Ford algorithm.*

Let $G = (V, E)$ be a directed and weighted graph with edge weights $w(e)$, which can be negative or positive or zero, for each edge $e \in E$. You may assume that G has no *negatively weighted* cycles. Additionally, each vertex is coloured either *red* or *blue*.

Given an integer $k \geq 2$, describe and analyse an $O(kn(km + n))$ algorithm that returns a path with the smallest weight, satisfying the following conditions:

- The path starts and ends on a vertex coloured red, and
- The number of vertices coloured blue in the path is divisible by k.

11. You are given a set M of n moving points on the plane. Each point p_i can be represented by a pair of coordinates (x_i, y_i) and a fixed velocity (u_i, v_i), representing its velocity along the x and y directions. A moving point p_i stops moving if one of two scenarios happen:

- p_i and p_j collide by occupying the same location at the same time, or
- p_i reaches a location that another point has previously occupied.

Every point will eventually stop moving by either running into another point's trail or by escaping to infinity. The complete pattern of trails left by the points is called the *motorcycle graph* of the point set M.

In this problem, assume that all points start on the x-axis; that is, $x_i = 0$ for each i. Describe and analyse an $O(n \log n)$ algorithm to compute the motorcycle graph of M.

12. Let $M = \{p_1, \ldots, p_n\}$ be a set of n discrete points on the $\mathbb{R}^2$ plane. A *Voronoi diagram of M* is a partition of the plane into n distinct regions, where region R_k denotes the set of all points in $\mathbb{R}^2$ that is closer to p_k than p_j for any $j \neq k$.

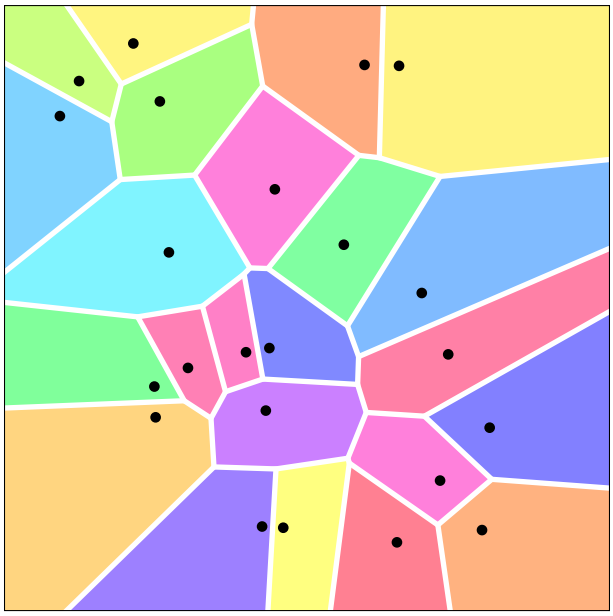

Figure 2: *The Voronoi diagram for a set of points.*

Assume that the input points are in general position; that is, no set of four points lie on the same circle and no three points are collinear. Given a set M of n points in general position, describe an $O(n \log n)$ algorithm that computes the Voronoi diagram of M.

13. Suppose you want to sketch the Manhattan skyline. You are given an array $M[1..n]$ of n rectangles (*buildings*) with their bottom edges on the x-axis. We can represent the ith rectangle by its left x-coordinate $L[i]$, its right x-coordinate $R[i]$, and its height $H[i]$. The *skyline* of M is given by the boundary of the set of points that are above the x-axis and outside *every* rectangle in M.

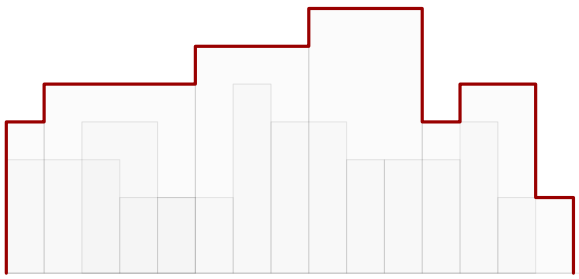

Figure 3: *The skyline of the given set of rectangles.*

Given an array of rectangles M, describe and analyse an $O(n \log n)$ algorithm that computes the skyline of M.

14. Let P be a simple polygon on n vertices.

    (a) We say that P is *y-monotone* if every horizontal line ℓ intersects P in at most two places. In other words, the intersection of P and ℓ is either a line segment, a point, or the empty set.

    Show that every simple polygon can be decomposed into a collection of disjoint y-monotone polygons. Describe and analyse an $O(n \log n)$ algorithm that computes this decomposition.

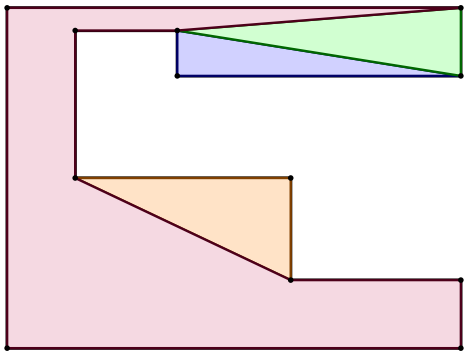

Figure 4: *A decomposition of a simple polygon where each part is a y-monotone polygon.*

(b) Let P be a simple polygon on n vertices. A *triangulation* of P is a partition of the interior of P into a collection of disjoint triangles. Let P be a y-monotone polygon. Describe and analyse an $O(n)$ algorithm that computes a triangulation of P.

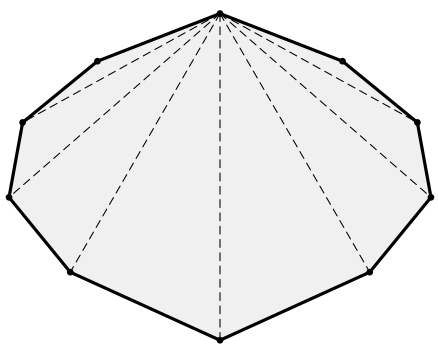

Figure 5: *A triangulation of a y-monotone polygon.*

(c) Hence, describe and analyse an $O(n \log n)$ algorithm that computes a triangulation of a simple polygon P. Note that P may or may not be y-monotone.

Note. *The same algorithm works for non-simple polygons, so it can be used to triangulate polygons with holes.*

15. **[Hard]** Let $S = \{s_1, \ldots, s_n\}$ be a set of disjoint line segments on the plane. We say that a set $R = \{r_1, \ldots, r_n\}$ of *horizontal* line segments is a *rectangulisation* of S if R satisfies the following properties:

- the endpoints of s_i and r_i have the same x-coordinate.

  Therefore, if a vertical line ℓ intersects s_i, then ℓ intersects r_i.

- if a vertical line ℓ intersects s_i and s_j, then $\ell \cap s_i$ is above $\ell \cap s_j$ if and only if $\ell \cap r_i$ is above $\ell \cap r_j$.

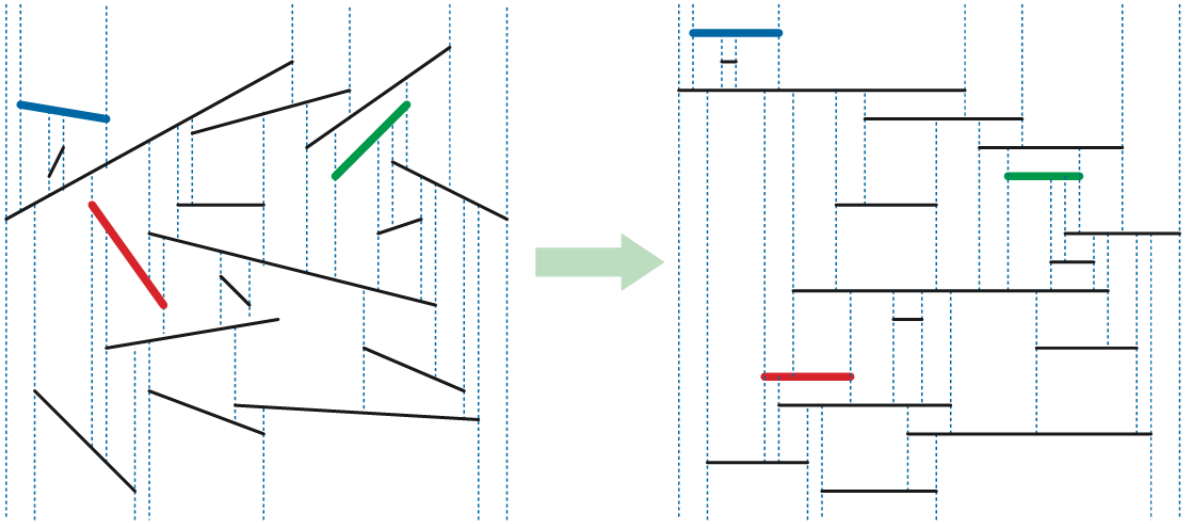

Figure 6: *A rectangulisation of a set of line segments.*

Describe and analyse a polynomial-time algorithm to compute a rectangulisation of S.

Hints

6. Start with the special case where $|X| = |Y| = 1$; what does this problem capture? Then generalise the ideas.
7. You cannot directly use Dijkstra's algorithm here; can you modify the graph so that Dijkstra's algorithm can be used?
8. The $1/k$th root is not particularly important for comparing the length of the paths. Since k is assumed to be constant, the time complexity is much easier to analyse; what happens if k is not constant?
9.
   (a) $h(v)$ is the weight of the shortest path from q to v; what does $h(u)+w(u \to v)$ represent?
   (b) The path becomes a telescoping series.
   (c) From the previous part, every path in $\hat{G}$ adds the same quantity.
10. Consider a layered graph; what should each layer represent?
11. Consider motorcycles moving to the right ($u_i > 0$) and motorcycles moving to the left ($u_i < 0$) separately. To find collisions among rightmoving motorcycles, consider collisions in order from left to right.
12. This is *Fortune's algorithm*. Consider a horizontal sweepline and define a *beach line*, which is an x-monotonic curve that follows behind the current sweep line. For a better idea of the algorithm, see this video.
13. Sweepline algorithm with events on the left and right vertices of each building; you should also maintain the current maximum height among all buildings that have been processed so far. As a follow up, can you do this in $O(n \log h)$, where h is the height of the maximum building?
14.
   (a) Sweepline algorithm with events on the vertices; a polygon is y-monotone if and only if it does not contain any of two types of vertices. Your goal is to figure out what types of vertices and then draw diagonals depending on the vertices.
   (b) It suffices to describe an $O(n)$ algorithm that triangulates a y-monotone polygon.
15. Show that S must contain at least one line segment with no other segments in S directly above it. Then sweep to find it in $O(n \log n)$ time to obtain an $O(n^2 \log n)$ algorithm. Can you do this in $O(n \log n)$ time?